

Token-Accounting Integrity: Characterizing Client-Side Under-Payment in LLM Metering Architectures

(anonymized for review)

Abstract

Usage-based billing is the default commercial model for LLM inference, yet the *accounting* pipeline that turns delivered tokens into money is itself a security boundary. Existing work on LLM billing studies two directions: a dishonest *provider* over-charging an honest client, and a third party inflating a *victim's* bill. We study the remaining direction — an honest provider and a dishonest *client* that seeks inference value while under-paying — and argue it is best understood not as a list of attacks but as a single accounting boundary spanning three architectural dimensions: **state synchronization** (is the authorize-and-decrement transition atomic?), **commitment timing** (is the charge irrevocable before value is delivered?), and **usage authority** (who determines the billed quantity?). We do not claim new attack primitives; the enabling mechanisms are known individually. Our contribution is to model these dimensions as transitions of a request-lifecycle state machine, build a reproducible vulnerable-by-construction testbed, and measure them under integrity gates that independently re-derive every reported number. Across 2,000+ instrumented requests we find that per-request leakage from commitment-timing and usage-authority defects is *invariant* to concurrency, whereas the synchronization defect is *created* by it — a structural distinction confirmed by an architectural ablation showing that different primitives close different properties: abort-inclusive finalization closes the integrity violation while reservation independently provides the solvency guarantee, and performing a server-side recount is insufficient unless it is also used as the billing basis. A local real-model experiment bounds the cost of the strongest defense at $\approx 0.015\%$ of end-to-end inference time, contradicting the apparent expense of the same defense measured against a zero-cost mock generator.

1 Introduction

Because the billed unit — the token — is produced by opaque, streamed, and long-running inference, the pipeline that authorizes, meters, and commits a charge is security-relevant. Recent work studies dishonest *providers* inflating that accounting against honest users [1, 2, 3], and availability/economic attacks that drain a victim’s budget [4, 5]. Both assume the client is honest or impersonated. We study the missing quadrant (Table 1): a legitimate but dishonest client that under-pays.

We are deliberately adversarial about our own novelty. Each enabling mechanism is already known in isolation: a deployed LLM gateway already refunds pre-consumed quota when a final usage record is missing after a client disconnect [6]; “never trust client input in a security decision” is CWE-807, and a metered-token proxy already mitigates an encoding-induced “silent zero-debit” [7]; and concurrent quota-decrement races are a classic web-race primitive with a recent CVE [8, 9]. Our contribution is therefore *systematization and measurement*, not discovery: we show these are three placements of one accounting boundary, localize each to a state transition, and quantify the integrity/performance trade-off of the defenses.

Table 1: Directions of LLM-billing security. This paper targets the last cell.

	Inflate cost	Evade cost
Provider dishonest	CoIn, Invisible Tokens, Token Inflation	—
Client dishonest	Denial-of-Wallet, LLMjacking	this paper

Contributions.

1. An **architectural taxonomy and accounting model** (§3) that localizes synchronization (B0), commitment timing (M1), and usage authority (M2) to specific transitions of a request-lifecycle state machine, with a single integrity property $V(r) \leq N(r)$.
2. A **reproducible client-side under-payment benchmark**: a vulnerable-by-construction gateway, attack harness, and defense implementations, with integrity gates that re-derive every reported number from a per-request ledger and fail the build on any discrepancy.
3. An **empirical characterization** of B0/M1/M2 across architectures, concurrency, token volume, abort position, price tier, tokenizer, and workload, including the negative result that M1/M2 leakage is concurrency-invariant.
4. An **architectural ablation** identifying which enforcement primitive closes which property — a stronger claim than “the defense works”.
5. An **integrity/performance trade-off** for authoritative recount, measured at three levels: standalone tokenizer, gateway end-to-end, and a local real-model experiment.

Research questions. **RQ1** How does the placement of synchronization, commitment, and usage authority affect client-side under-payment? **RQ2** How much inference value can a dishonest client obtain without authoritative billing? **RQ3** Which failures are visible to the application’s own logs and reconciliation? **RQ4** Which enforcement primitives close each violation, and at what cost?

2 Threat Model

Honest provider; dishonest client with valid credentials. Concretely, the attacker controls only: request payloads and any client-declared usage fields; permitted headers; request concurrency; the client connection lifecycle (it may abort at any moment); retries; and observation of its own responses and balance. It *cannot* compromise TLS, access the database or Redis, read the gateway filesystem, compromise the inference backend, forge authenticated upstream/provider metadata, or alter server code. Every listed capability is exercised by the harness; none is granted that we do not use. This is an application-layer adversary, not a symbolic cryptographic one.

Success on request r means receiving value $V(r)$ while the account is net-charged less. All experiments run against a local testbed; no third-party system is contacted.

3 Accounting State Model

We model a metered inference request as a state machine so that each mechanism can be localized to a specific transition, establishing that the taxonomy is *architectural* rather than a list of unrelated implementation bugs.

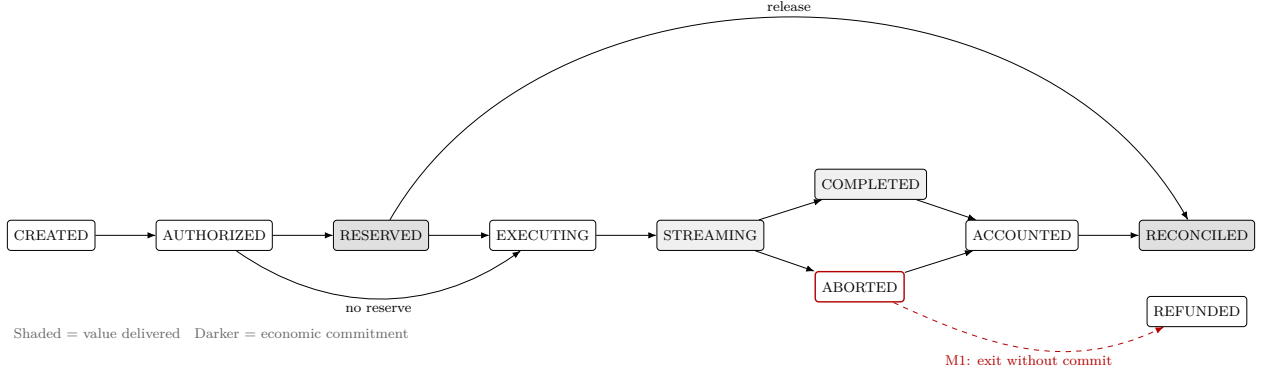


Figure 1: Request lifecycle. Value exposure begins at the first delivered token (STREAMING); economic commitment is final only at RECONCILED. The interval between them is the exposure window that M1 exploits. B0 corrupts the AUTHORIZED→RESERVED transition (non-atomic); M2 corrupts the *data* used at ACCOUNTED.

3.1 Formal model

For a request r let $U_{\text{true}}(r)$ be the usage actually produced, $U_{\text{bill}}(r)$ the usage billed, and $P(\cdot)$ the (exact-decimal) pricing function. Write $C(r) = P(U_{\text{true}}(r))$ for the true cost, $V(r)$ for the value actually delivered to the client, $\text{Res}(r)$ for any reservation, $D(r)$ for committed debits, $F(r)$ for refunds, and

$$N(r) = D(r) - F(r)$$

for the *net committed debit*. Under-payment and the integrity property are

$$\text{Leak}(r) = V(r) - N(r), \quad \text{Integrity}(r) : V(r) \leq N(r).$$

Because legitimate reconciliation exists, integrity is an inequality on the *net* debit; a refund is legitimate only when it releases the unconsumed portion,

$$F(r) \leq \text{Res}(r) - V(r).$$

At trial level the harness independently verifies conservation,

$$\sum_{r \in T} N(r) = B_{\text{initial}} - B_{\text{final}},$$

and fails the build on any divergence.

3.2 Mechanism localization

B0 (synchronization). If two concurrent requests read the same authorization state B and both commit a decrement derived from it, one decrement is lost: $\sum_i N(r_i) < \sum_i V(r_i)$. This requires ≥ 2 concurrent transactions, which is why B0 leaks nothing at concurrency 1.

M1 (commitment timing). If $V(r) > 0$ in STREAMING but the exit path taken (ABORTED) does not reach RECONCILED with $N(r) \geq V(r)$, then $\text{Leak}(r) = V(r) > 0$. The defect is in the transition graph, not in a shared resource, hence it is concurrency-invariant.

Table 2: Structural distinction between the three mechanisms.

Mechanism	Defective element	Needs concurrency?	Closed by
B0	transition atomicity	yes (≥ 2)	atomic compare-and-decrement
M1	transition graph (abort path)	no	abort-inclusive finalization
M2	data authority at ACCOUNTED	no	server-authoritative recount

M2 (usage authority). If the billing basis is client-controlled, $U_{\text{bill}} = T(U_{\text{true}})$ for an attacker transformation T , then $\text{Leak}(r) = P(U_{\text{true}}) - P(T(U_{\text{true}})) > 0$ whenever $P \circ T < P$. Authority is a per-request data-provenance property, hence also concurrency-invariant.

This predicts the empirical result: B0 leakage is *created* by concurrency, while M1/M2 per-request leakage is invariant to it.

4 Testbed and Methodology

FastAPI gateway, PostgreSQL 16 (READ COMMITTED), Redis 7, a deterministic mock generator for reproducibility, and real tokenizers for the recount path; Docker Compose on one commodity host (16 logical CPUs, 16 GB). Money is exact decimal (NUMERIC(18,6)). Category-wise pricing over {input, output, cached, reasoning} in three tiers; leakage *efficiency* (fraction of true value evaded) is the price-invariant headline metric, and absolute dollars are labelled scenario values.

Integrity gates. Every summarizer independently recomputes each record’s leak from primitives, checks the invariant flag for consistency, and verifies ledger conservation $\sum_r N(r) = B_{\text{init}} - B_{\text{final}}$ *without* trusting the application’s own bookkeeping. A violation fails the build. We validate the gates adversarially by fault injection: corrupting a leak value, a debit, a served flag, a balance, an invariant flag, or deleting a ledger row must each be detected (12/12 detected), alongside metamorphic properties (10/10; see `results/processed/metamorphic_report.json`).

Statistics. All reported statistics are produced by the standalone script `statistical_analysis.py` (scipy/statsmodels), which writes `stats_output.txt`; the paper cites that file. Many conditions here are deterministic by construction (identical input, identical code path), giving exactly zero variance; for those we report the constant and complete separation rather than a p-value, and reserve nonparametric tests and bootstrap intervals for quantities that genuinely vary (latency, throughput, real-model timing).

5 Results

5.1 B0 — synchronization (known baseline)

The non-atomic check-then-decrement leaks only when concurrency ≥ 2 : mean \$-leak is 0 at $c=1$ and grows exactly linearly, slope 0.099 per additional concurrent request with $R^2 = 1.0000$ ($p = 1.9 \times 10^{-50}$). The hardened atomic compare-and-decrement leaks \$0 at every level (Table 3, Fig. 5).

5.2 M1 — commitment timing

Leakage tracks the value delivered before the (missing) commit: it rises monotonically with abort position and collapses to zero at completion, when the debit finally fires (Fig. 2, Table 4). Both vulnerable architectures — “debit only after completion” and “refund the whole reservation on abort” — are economically identical.

Concurrency is not the mechanism. Holding request volume constant, leak per request is invariant across $c \in \{1, 5, 20, 50, 100\}$ to four decimal places, and request-ASR is exactly 1.0 (vulnerable) or 0.0 (safe) at every level (Table 5, Fig. 3). The honest-client control arm leaks zero everywhere. M1 is a per-request accounting-design defect, not a race.

5.3 M2 — usage authority

Client-authoritative billing leaks for most manipulations, and *which* manipulation succeeds depends on the billing basis: under category pricing a total/subtotal mismatch is inert, while under flat-total pricing it becomes the strongest attack (Table 6, Fig. 4). Every server-authoritative architecture drives leakage efficiency to 0.000 across all manipulations, and is likewise concurrency-invariant (Table 7).

5.4 Architectural ablation: which primitive closes what?

Removing one primitive at a time (Tables 8, 9) yields the paper’s sharpest architectural result:

- **M1:** *abort-inclusive finalization* closes the integrity violation — with or without a reservation. *Reservation* closes a *different* property: without it the balance goes negative under a constrained budget (solvency), even though integrity holds. Neither primitive substitutes for the other.
- **M2:** performing a recount is *not* sufficient. The `client_logged` architecture computes an authoritative recount and still leaks at efficiency 0.595, because billing still uses the client’s number. The recount must be *used as the billing basis*.

5.5 Lifecycle failure modes

Beyond the scripted attack, we inject nine lifecycle failures (disconnect at $t=0$ and mid-stream, truncated stream, settle-window race, missing / delayed / duplicated usage metadata, and a recount-engine failure). Every invariant violation falls on a vulnerable architecture; the hardened architectures survive all of them, and a recount-engine failure is *fail-closed* — the request is rejected and nothing is served or billed (Table 10).

5.6 Detectability

Detection is a second defense axis (Table 11). Under `client`-authoritative billing the leak is invisible (D0): the ledger faithfully records the client’s lie. Merely *logging* a server recount moves the same leak to D1 — recoverable by a post-hoc reconciliation pass — at negligible cost, and reconciling in-request reaches D3. This gives operators who cannot re-architect billing an immediate, cheap intermediate step.

Table 3: B0 baseline: credit-decrement race (30 reps/cell).

posture	conc	served	\$-leak	trial ASR	req ASR	detect
hardened	1	1.0	0.0000	0.00	0.000	none
hardened	2	2.0	0.0000	0.00	0.000	none
hardened	5	5.0	0.0000	0.00	0.000	none
hardened	10	10.0	0.0000	0.00	0.000	none
hardened	20	10.0	0.0000	0.00	0.000	none
hardened	30	10.0	0.0000	0.00	0.000	none
hardened	50	10.0	0.0000	0.00	0.000	none
vulnerable	1	1.0	0.0000	0.00	0.000	none
vulnerable	2	2.0	0.0990	1.00	0.500	partial
vulnerable	5	5.0	0.3960	1.00	0.800	partial
vulnerable	10	10.0	0.8910	1.00	0.900	partial
vulnerable	20	20.0	1.8810	1.00	0.950	partial
vulnerable	30	30.0	2.8710	1.00	0.967	partial
vulnerable	50	50.0	4.8510	1.00	0.980	partial

5.7 Cost of authoritative recount (RQ4)

We measure the strongest defense at three levels, because they answer different questions.

1. **Standalone tokenizer** (Fig. 6): cost is linear in input length (log-log slope 0.999–1.14, $R^2 \geq 0.998$), so it budgets as $\text{tokens} \times \text{a per-token constant}$. Engine choice matters by 2.9–3.6 $\times$ at 16,384 tokens; at a fixed token count, workload changes cost by up to 3.2 $\times$.
2. **Gateway end-to-end with a *mock* generator**: recount is nearly free at $\leq 1,024$ tokens ($\geq 0.96\times$ throughput) but costs heavily at 16,384 tokens (0.05–0.76 $\times$). Tokenization time explains this (Spearman $\rho = -0.868$, $p = 1.3 \times 10^{-5}$).
3. **Local real-model experiment** (external validity): with *real* autoregressive generation, the recount is **0.012%–0.016%** of end-to-end time at every context size (bootstrap 95% CIs in `stats_output.txt`); measured throughput ratios scatter around 1.0 (0.89–1.05) with signs that flip across workloads, i.e. within timing noise.

The apparent expense in (2) is an artifact of the mock generator costing *nothing*: when generation is free, tokenization is the whole request. Under real inference the same defense is negligible. We report all three rather than the most convenient one.

6 Discussion and Threats to Validity

The mechanisms are reframings of known issues, not new primitives; we state this throughout and use bounded language for the literature gap (“we found no prior work in our targeted corpus”,

Table 4: M1 metering-commit timing: leak vs. abort (medium tier, concurrency=1).

architecture	abort%	delivered	leak/req	req ASR	inv.viol	detect
pre_debit	0	1	0.0000	0.00	0.00	n/a
pre_debit	25	17	0.0000	0.00	0.00	n/a
pre_debit	50	34	0.0000	0.00	0.00	n/a
pre_debit	75	52	0.0000	0.00	0.00	n/a
pre_debit	90	62	0.0000	0.00	0.00	n/a
pre_debit	100	69	0.0000	0.00	0.00	n/a
reserve_reconcile	0	1	0.0000	0.00	0.00	n/a
reserve_reconcile	25	17	0.0000	0.00	0.00	n/a
reserve_reconcile	50	34	0.0000	0.00	0.00	n/a
reserve_reconcile	75	52	0.0000	0.00	0.00	n/a
reserve_reconcile	90	62	0.0000	0.00	0.00	n/a
reserve_reconcile	100	69	0.0000	0.00	0.00	n/a
post_completion	0	1	0.0045	1.00	1.00	D0
post_completion	25	17	0.0285	1.00	1.00	D0
post_completion	50	34	0.0540	1.00	1.00	D0
post_completion	75	52	0.0810	1.00	1.00	D0
post_completion	90	62	0.0960	1.00	1.00	D0
post_completion	100	69	0.0000	0.00	0.00	n/a
reserve_refund_on_abort	0	1	0.0045	1.00	1.00	D0
reserve_refund_on_abort	25	17	0.0285	1.00	1.00	D0
reserve_refund_on_abort	50	34	0.0540	1.00	1.00	D0
reserve_refund_on_abort	75	52	0.0810	1.00	1.00	D0
reserve_refund_on_abort	90	62	0.0960	1.00	1.00	D0
reserve_refund_on_abort	100	69	0.0000	0.00	0.00	n/a

never “nobody has studied this”). Principal threats: the generator is a mock in all but the external-validity experiment, and that experiment uses a small model on CPU — it bounds a *ratio*, and says nothing about any commercial provider; the gateway runs a single uvicorn worker, so its recount numbers are an upper bound; measurements are single-node over loopback, and high-concurrency gateway ratios are noisy (repeat spread up to 17%), so only low-concurrency columns are treated as defensible; and the price tiers are synthetic, which is why leakage efficiency rather than dollars is the headline. A full list is in `paper/threats_to_validity.md`.

7 Conclusion

Client-side under-payment in LLM metering is best modelled not as a list of attacks but as one accounting boundary with three placements: synchronization, commitment timing, and usage authority. Localizing each to a state transition predicts — and our measurements confirm — that only the synchronization defect is concurrency-created, and that different primitives close integrity and solvency. The defenses are ordinary; the contribution is knowing precisely which one is load-bearing, and what it costs.

Table 5: M1: leak PER REQUEST vs concurrency (request volume held constant, attacker arm).

architecture	c=1	c=5	c=20	c=50	c=100
post_completion	0.0528	0.0528	0.0529	0.0529	0.0528
pre_debit	0.0000	0.0000	0.0000	0.0000	0.0000
reserve_reconcile	0.0000	0.0000	0.0000	0.0000	0.0000
reserve_refund_on_abort	0.0528	0.0528	0.0528	0.0528	0.0528

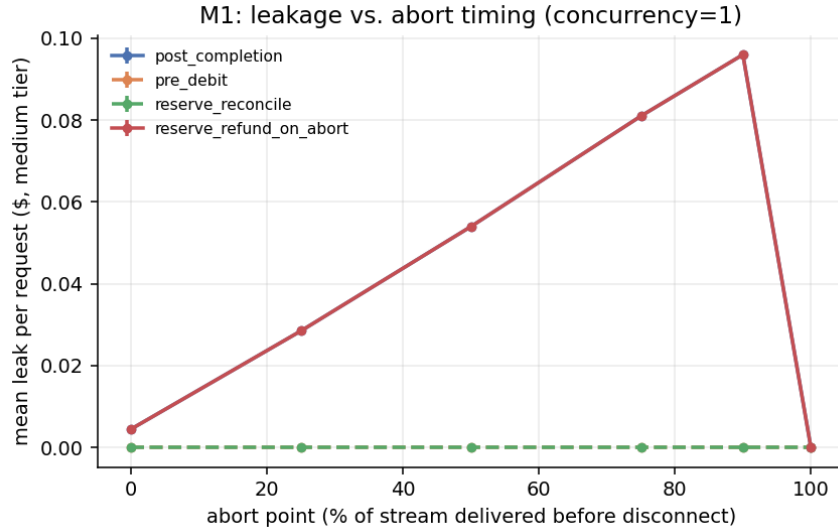


Figure 2: M1: leakage vs. abort position. Vulnerable architectures peak just before the commit and collapse to \$0 at completion; safe architectures stay at \$0.

References

- [1] G. Sun et al. CoIn: Counting the Invisible Reasoning Tokens in Commercial Opaque LLM APIs. arXiv:2505.13778, 2025.
- [2] G. Sun et al. Invisible Tokens, Visible Bills. arXiv:2505.18471, 2025.
- [3] S. Hoque et al. Token Inflation: How Dishonest Providers Can Overcharge for LLM Usage. arXiv:2605.30040, 2026.
- [4] OWASP. LLM10: Unbounded Consumption (Denial of Wallet). OWASP Top 10 for LLM Applications, 2025.
- [5] Sysdig Threat Research Team. LLMjacking: stolen cloud credentials used for LLM inference, 2024.
- [6] QuantumNous/new-api issue #5235: streaming requests may refund all pre-consumed quota when final usage is missing after client disconnect, 2026.
- [7] lightninglabs/aperture PR #247: metered draw-down of prepaid L402 tokens; strip `Accept-Encoding` to prevent a silent zero-debit, 2026.
- [8] J. Kettle. The single-packet attack: making remote race-conditions ‘local’. PortSwigger Research, 2023.

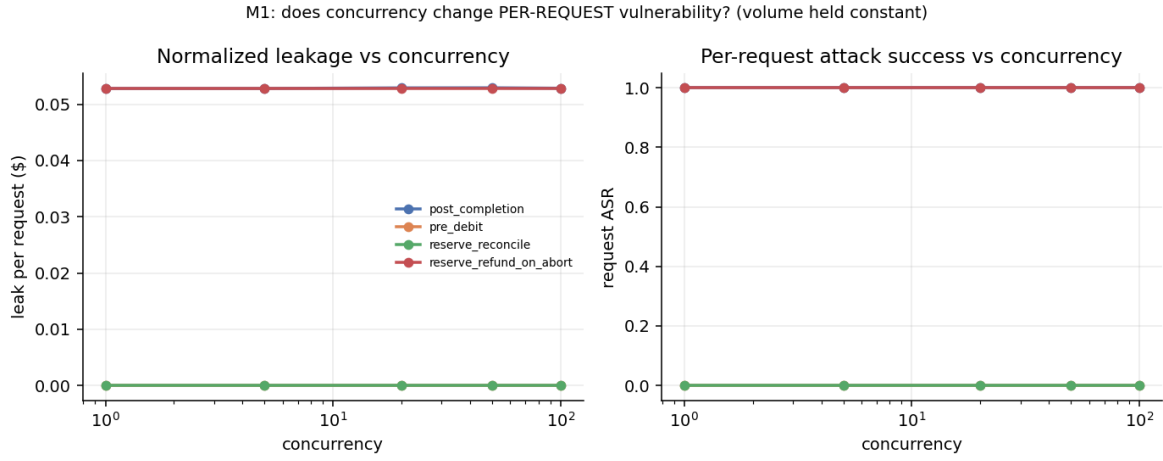


Figure 3: M1 negative result: with request volume held constant, per-request leakage and request-ASR are invariant to concurrency.

[9] CVE-2026-31873: Tyk API Gateway non-atomic quota check/decrement race, 2026.

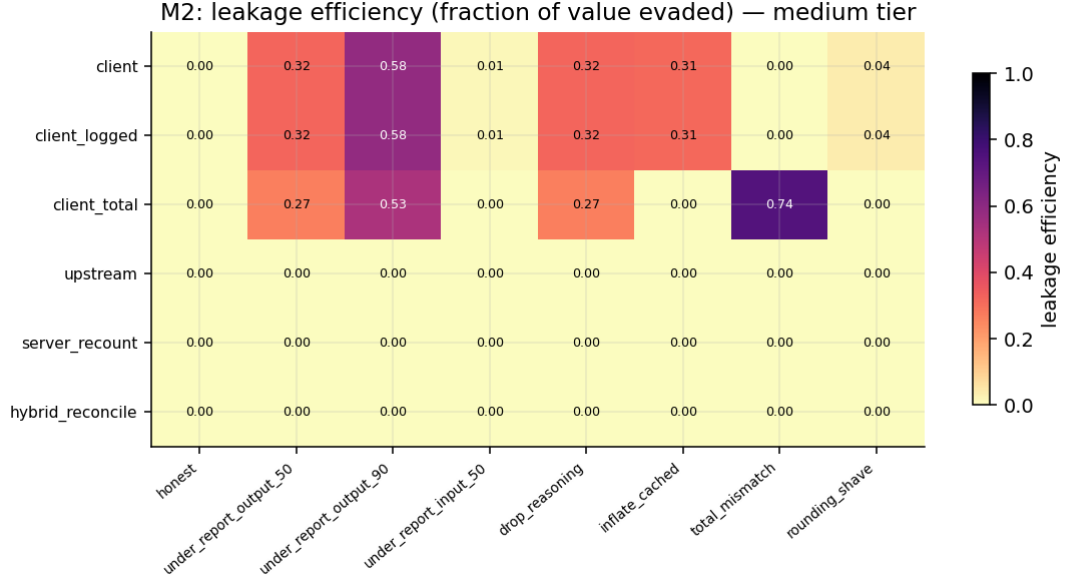


Figure 4: M2: leakage efficiency by architecture $\times$ manipulation. Server-authoritative rows are uniformly zero; the exploitable set differs between category and flat-total pricing.

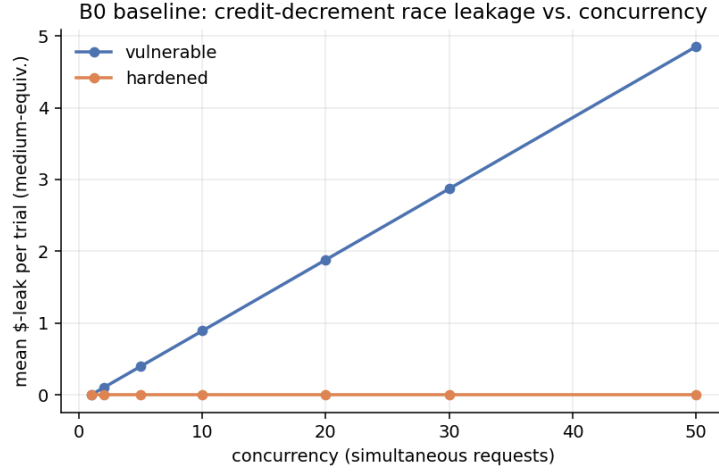


Figure 5: B0 baseline: leakage is created by concurrency (contrast with M1/M2).

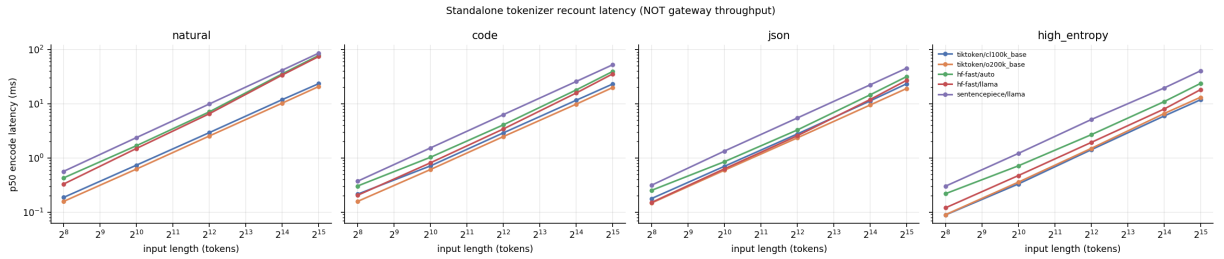


Figure 6: Standalone tokenizer recount latency by engine, workload, and input length (log-log). This is *not* a gateway throughput result.

Table 6: M2 usage-record authority: leakage efficiency by manipulation (medium tier, concurrency=1).

architecture	manipulation	leak eff.	req ASR	inv.viol	detect
client	honest	0.00	0.00	0.00	n/a
client	under_report_output_50	0.32	1.00	1.00	D0
client	under_report_output_90	0.58	1.00	1.00	D0
client	under_report_input_50	0.01	1.00	1.00	D0
client	drop_reasoning	0.32	1.00	1.00	D0
client	inflate_cached	0.31	1.00	1.00	D0
client	total_mismatch	0.00	0.00	0.00	n/a
client	rounding_shave	0.04	1.00	1.00	D0
client_logged	honest	0.00	0.00	0.00	n/a
client_logged	under_report_output_50	0.32	1.00	1.00	D1
client_logged	under_report_output_90	0.58	1.00	1.00	D1
client_logged	under_report_input_50	0.01	1.00	1.00	D1
client_logged	drop_reasoning	0.32	1.00	1.00	D1
client_logged	inflate_cached	0.31	1.00	1.00	D1
client_logged	total_mismatch	0.00	0.00	0.00	n/a
client_logged	rounding_shave	0.04	1.00	1.00	D1
client_total	honest	0.00	0.00	0.00	n/a
client_total	under_report_output_50	0.27	1.00	1.00	D0
client_total	under_report_output_90	0.53	1.00	1.00	D0
client_total	under_report_input_50	0.00	0.00	0.00	n/a
client_total	drop_reasoning	0.27	1.00	1.00	D0
client_total	inflate_cached	0.00	0.00	0.00	n/a
client_total	total_mismatch	0.74	1.00	1.00	D0
client_total	rounding_shave	0.00	0.00	0.00	n/a
upstream	honest	0.00	0.00	0.00	n/a
upstream	under_report_output_50	0.00	0.00	0.00	n/a
upstream	under_report_output_90	0.00	0.00	0.00	n/a
upstream	under_report_input_50	0.00	0.00	0.00	n/a
upstream	drop_reasoning	0.00	0.00	0.00	n/a
upstream	inflate_cached	0.00	0.00	0.00	n/a
upstream	total_mismatch	0.00	0.00	0.00	n/a
upstream	rounding_shave	0.00	0.00	0.00	n/a
server_recount	honest	0.00	0.00	0.00	n/a
server_recount	under_report_output_50	0.00	0.00	0.00	n/a
server_recount	under_report_output_90	0.00	0.00	0.00	n/a
server_recount	under_report_input_50	0.00	0.00	0.00	n/a
server_recount	drop_reasoning	0.00	0.00	0.00	n/a
server_recount	inflate_cached	0.00	0.00	0.00	n/a
server_recount	total_mismatch	0.00	0.00	0.00	n/a
server_recount	rounding_shave	0.00	0.00	0.00	n/a
hybrid_reconcile	honest	0.00	0.00	0.00	n/a
hybrid_reconcile	under_report_output_50	0.00	0.00	0.00	n/a
hybrid_reconcile	under_report_output_90	0.00	0.00	0.00	n/a
hybrid_reconcile	under_report_input_50	0.00	0.00	0.00	n/a
hybrid_reconcile	drop_reasoning	0.00	0.00	0.00	n/a
hybrid_reconcile	inflate_cached	0.00	0.00	0.00	n/a

Table 7: M2: leakage efficiency vs concurrency (attacker arm; volume held constant).

architecture	c=1	c=5	c=20	c=50	c=100
client	0.228	0.228	0.228	0.228	0.228
client_logged	0.228	0.228	0.228	0.228	0.228
client_total	0.257	0.257	0.257	0.257	0.257
hybrid_reconcile	0.000	0.000	0.000	0.000	0.000
server_recount	0.000	0.000	0.000	0.000	0.000
upstream	0.000	0.000	0.000	0.000	0.000

Table 8: M1 ablation: abort-inclusive finalization closes the integrity violation; reservation independently provides the solvency (funds) guarantee. Both are needed for a correct design.

architecture	reservation	abort-finalization	worst leak/req	integrity	served (budget 2)	solvency
pre_debit	yes	n/a (pre-committed)	-0.0090	held	2/6	ok
reserve_reconcile	yes	yes	+0.0000	held	2/6	ok
reserve_refund_on_abort	yes	no	+0.0845	VIOLATED	6/6	ok
no_reserve_settle	no	yes	+0.0000	held	6/6	NEGATIVE
post_completion	no	no	+0.0845	VIOLATED	6/6	ok

Table 9: M2 ablation: performing a recount is not sufficient; it must be used as the billing basis. client_logged computes the recount and still leaks.

architecture	recount performed	recount used for billing	worst leak eff.	integrity	detection
client	no	no	0.595	VIOLATED	D0
client_logged	yes	no	0.595	VIOLATED	D1
client_total	no	no	0.745	VIOLATED	D0
server_recount	yes	yes	0.000	held	n/a
hybrid_reconcile	yes	yes	0.000	held	n/a
upstream	no	no	0.000	held	n/a

Table 10: Lifecycle failure injection. Every invariant violation occurs on a vulnerable architecture; the hardened architectures survive all injected failures, and a recount-engine failure is fail-closed.

id	failure injected	architecture	delivered	net debit	refund	leakage	invariant
F1	client disconnect mid-stream	post_completion	yes	0.0000	0.0000	0.2375	VIOLATED
F1	client disconnect mid-stream	reserve_reconcile	yes	0.2375	0.2175	0.0000	HELD
F2	client disconnect before first token	post_completion	yes	0.0000	0.0000	0.0275	VIOLATED
F2	client disconnect before first token	reserve_reconcile	yes	0.0275	0.4275	0.0000	HELD
F8	partial/truncated stream near completion	post_completion	yes	0.0000	0.0000	0.4325	VIOLATED
F8	partial/truncated stream near completion	reserve_reconcile	yes	0.4325	0.0225	0.0000	HELD
F9	audit read before settle grace period	post_completion	yes	0.0000	0.0000	0.2375	VIOLATED
F9	audit read before settle grace period	reserve_reconcile	yes	0.2375	0.2175	0.0000	HELD
F5	missing/unknown usage metadata	client	yes	0.6725	0.0000	0.0000	HELD
F5	missing/unknown usage metadata	server_recount	yes	0.6725	0.0000	0.0000	HELD
F7	duplicate usage metadata (replayed request_i	client	yes	0.2750	0.0000	0.3975	VIOLATED
F7	duplicate usage metadata (replayed request_i	server_recount	yes	0.6725	0.0000	0.0000	HELD
F6	delayed/out-of-order usage metadata	client	yes	0.4550	0.0000	0.2175	VIOLATED
F6	delayed/out-of-order usage metadata	server_recount	yes	0.6725	0.0000	0.0000	HELD
F4	tokenizer/recount engine unavailable	bench/server_recount	no	0.0000	0.0000	0.0000	HELD

Table 11: Detectability matrix. D0 invisible; D1 only via post-hoc reconciliation; D2 logged discrepancy; D3 prevented or corrected in real time. *B0’s final balance alone cannot reveal the loss because the ledger and the balance are both under-decremented consistently with the surviving write.

mech	architecture	req logs	usage ledger	reconciliation	final balance	real-time	level	detection latency
B0	vulnerable (non-atomic)	no	yes	yes	no*	no	D1	until a reconciliation pass
B0	hardened (atomic CAS)	n/a	n/a	n/a	n/a	yes	D3	0 (prevented)
M1	post_completion	partial	no	no	no	no	D0	never (no record is written)
M1	reserve_refund_on_abort	partial	yes	yes	yes	no	D1	until a reconciliation pass
M1	reserve_reconcile	n/a	n/a	n/a	n/a	yes	D3	0 (prevented)
M2	client	no	no	no	no	no	D0	never (ledger records the lie)
M2	client_logged	no	yes	yes	no	no	D1	until a reconciliation pass
M2	server_recount	n/a	n/a	n/a	n/a	yes	D3	0 (prevented)
M2	hybrid_reconcile	yes	yes	yes	n/a	yes	D3	0 (detected and corrected in-request)

Table 12: Architecture safety summary across all attacks and tiers.

mech	architecture	verdict	max req ASR	best detect
M1	pre_debit	safe	0.00	D3
M1	reserve_reconcile	safe	0.00	D3
M1	post_completion	LEAKS	1.00	D0
M1	reserve_refund_on_abort	LEAKS	1.00	D0
M2	client	LEAKS	1.00	D0
M2	client_logged	LEAKS	1.00	D1
M2	client_total	LEAKS	1.00	D0
M2	upstream	safe	0.00	D3
M2	server_recount	safe	0.00	D3
M2	hybrid_reconcile	safe	0.00	D3